

RAPORT TECHNICZNY: KOMPLEKSOWA SPECYFIKACJA SYSTEMU KOMPONENTÓW UI – MODUŁ PRZYCISKÓW (ATOMIC DESIGN)

1. Wstęp: Filozofia Atomowa w Kontekście Nowoczesnego Interfejsu

Współczesna inżynieria oprogramowania oraz projektowanie interfejsów użytkownika (UI) ewoluowały z podejścia opartego na stronach (page-based) w stronę systemów zorientowanych na komponenty. W paradygmacie **Atomic Design**, sformułowanym przez Brada Frosta, przycisk (button) nie jest trywialnym prostokątem z tekstem, lecz fundamentalnym „atomem” – niepodzielną jednostką funkcjonalną, która stanowi o interaktywności całego systemu. Niniejszy raport stanowi wyczerpującą, ekspercką analizę i specyfikację techniczną modułu przycisków, opracowaną w odpowiedzi na zapotrzebowanie stworzenia unikalnego systemu wizualnego opartego na estetyce „Premium Modern”.

Projekt ten stawia przed architektami systemu wyzwanie szczególne: pogodzenie luksusowej, nasyconej palety kolorystycznej (złoto, ciemny turkus, fiolet) z rygorystycznymi standardami dostępności cyfrowej (Accessibility, WCAG 2.1) oraz wydajności renderowania (Web Performance). Analiza ta wykracza poza powierzchowny opis stylów CSS, zagłębiając się w psychofizjologię percepcji koloru, typograficzną inżynierię kroju **Mukta Malar**, matematykę krzywych Bezierra przy animacjach oraz architekturę unikania przesunięć układu (CLS) podczas asynchronicznych stanów ładowania.

Celem dokumentu jest dostarczenie zespołom deweloperskim i projektowym kompletnego „źródła prawdy” (Single Source of Truth), które eliminuje niejasności implementacyjne i gwarantuje, że finalny produkt będzie nie tylko estetycznie spójny, ale również technicznie bezbłędny, skalowalny i dostępny dla wszystkich grup użytkowników, niezależnie od ich ograniczeń percepcyjnych czy sprzętowych.

2. Typografia: Analiza Strukturalna i Implementacja Kroju Mukta Malar

Fundamentem komunikacji wewnątrz komponentu przycisku jest typografia. Wybór kroju pisma determinuje nie tylko czytelność, ale również charakter emocjonalny interfejsu. Zgodnie z wymaganiami projektowymi, system opiera się na kroju **Mukta Malar** w odmianie SemiBold. Decyzja ta, choć na pierwszy rzut oka estetyczna, niesie ze sobą głębokie implikacje techniczne i percepcyjne, które wymagają szczegółowego omówienia.

2.1. Charakterystyka Morfologiczna i Dziedzictwo Kulturowe

Mukta Malar to otwartoźródłowy (licencja SIL Open Font License), humanistyczny krój bezszeryfowy, zaprojektowany przez Aadarsha Rajana w ramach inicjatywy Ek Type. Jest to część szerszej super-rodziny „Mukta”, stworzonej jako projekt wieloskrpity (multi-script), mający na celu harmonizację wizualną różnorodnych systemów pisma indyjskiego (w tym przypadku tamilskiego) z alfabetem łacińskim.

Analiza morfologii liter łacińskich w Mukta Malar ujawnia cechy kluczowe dla interfejsów użytkowych (UI):

- **Mono-linearność:** Litery charakteryzują się relatywnie stałą grubością kreski (stroke width). W kontekście przycisków, gdzie tekst jest często renderowany na silnie kontrastowym lub nasyconym tle (złoto, fiolet), mono-linearność zapobiega zanikaniu cieńszych elementów litery, co jest częstym problemem w krojach o wysokim kontraście (np. Didone) przy małych stopniach pisma.
- **Otwarty Dukt (Open Apertures):** Mukta posiada szerokie otwarcia w literach takich jak „c”, „e” czy „s”. Jest to cecha krytyczna dla czytelności na ekranach o niższej rozdzielczości oraz w trudnych warunkach oświetleniowych, zapobiegając optycznemu zlewaniu się kształtów.
- **Humanistyczny Szkielet:** Mimo nowoczesnego sznytu, krój zachowuje organiczne proporcje nawiązujące do pisma ręcznego, co łagodzi techniczny chłód interfejsu i wpisuje się w pożądaną stylistykę „nowoczesnej elegancji”.

2.2. Inżynieria Wag: Dlaczego SemiBold?

Wymagania specyfikują użycie wagi **SemiBold** (wartość numeryczna 600) dla etykiet przycisków. Analiza dostępnych wag rodziny Mukta Malar (ExtraLight, Light, Regular, Medium, SemiBold, Bold, ExtraBold) potwierdza, że wybór ten jest optymalny z punktu widzenia zjawiska irradacji.

Złote tło (Primary) charakteryzuje się wysoką luminancją. Gdy ciemny tekst (turkus) jest umieszczony na jasnym, świetlistym tle, zachodzi zjawisko optyczne, w którym tło „wżera się” w litery, sprawiając, że wydają się one cieńsze niż w rzeczywistości. Standardowa waga Regular (400) mogłaby w tych warunkach utracić wyrazistość i stać się nieczytelna. Waga SemiBold (600) kompensuje ten efekt, dodając wystarczającą masę optyczną, aby etykieta przycisku pozostała dominantą wizualną, jednocześnie nie popadając w ciężkość wagi Bold (700), która mogłaby zaburzyć elegancję kompozycji.

2.3. Strategia Ładowania Fontów i Metryki Wertykalne

Zastosowanie niestandardowego fontu (Web Font) wiąże się z ryzykiem opóźnień w renderowaniu tekstu (FOIT - Flash of Invisible Text) lub niepożądanych przesunięć układu (FOUT - Flash of Unstyled Text). Biorąc pod uwagę dostępność pakietów NPM takich jak [@fontsource/mukta-malar](https://www.npmjs.com/package/@fontsource/mukta-malar), rekomenduje się strategię self-hosting zamiast korzystania z CDN Google Fonts, co zwiększa kontrolę nad wydajnością i prywatnością.

Istotnym aspektem technicznym są metryki wertykalne kroju. Fonty projektowane z myślą o pismach indyjskich (jak Tamil) często posiadają wyższe linie wydłużeń górnych (ascenders) i dolnych (descenders) niż standardowe kroje łacińskie, aby pomieścić znaki diakrytyczne.

Wymaga to precyzyjnego dostrojenia właściwości line-height oraz padding wewnątrz przycisku, aby optycznie wyśrodkować tekst. Standardowe centrowanie CSS (Flexbox/Grid) może nie wystarczyć, jeśli fizyczna linia bazowa fontu jest przesunięta. W przypadku Mukta Malar, testy wskazują na konieczność stosowania line-height: 1.2 do 1.5 w zależności od rozmiaru, aby

zachować balans.

Parametr Typograficzny	Wartość / Ustawienie	Uzasadnienie
Rodzina (Family)	Mukta Malar	Spójność z systemem designu, wsparcie wielojęzyczne.
Waga (Weight)	600 (SemiBold)	Kompensacja kontrastu tła, hierarchia wizualna.
Styl (Style)	Normal	Kursywa nie jest zalecana dla etykiet przycisków (zmniejsza czytelność).
Tracking (Letter-spacing)	+0.02em (procentowo)	Nieznaczne zwiększenie światła dla małych liter w Uppercase/Sentence Case.
Renderowanie	-webkit-font-smoothing: antialiased	Poprawa ostrości krawędzi na ciemnoturkusowym tekście.

3. System Kolorystyczny: Alchemia Złota i Dostępność

Kolor w systemie UI pełni podwójną rolę: buduje tożsamość marki i komunikuje stan interfejsu. Zestawienie Złota (Primary Background) i Ciemnego Turkusu (Primary Text) jest odważne i rzadko spotykane w standardowych bibliotekach UI. Wymaga ono zatem rygorystycznej weryfikacji pod kątem norm WCAG 2.1, aby zapewnić, że estetyka nie wykluczy użytkowników z wadami wzroku.

3.1. Definicja Wartości Kolorystycznych (Tokenizacja)

Aby przełożyć opis słowny „tło złote, tekst ciemnoturkusowy” na precyzyjny język inżynierii, musimy zdefiniować konkretne wartości HEX, które spełnią wymogi kontrastu.

Kolor Primary Background (Złoto): Czyste złoto w modelu cyfrowym jest problematyczne. Standardowy kolor „Gold” (#FFD700) bywa zbyt jaskrawy. Kolory metaliczne są trudne do oddania płaskim kolorem (flat design). Rekomendacja: **#D4AF37** (Metallic Gold). Jest to odcień zbalansowany, elegancki, nie wpadający w neonową żółć, a jednocześnie posiadający wystarczającą luminancję, by być tłem.

Kolor Primary Text (Ciemny Turkus): Musi on tworzyć maksymalny kontrast ze złotem. Rekomendacja: **#004D40** (Deep Teal / Dark Turquoise). Jest to kolor o bardzo niskiej luminancji, bliski czerni, ale zachowujący bogatą, morską barwę.

Analiza Kontrastu (WCAG 2.1): Narzędzia analityczne pozwalają nam obliczyć współczynnik kontrastu (Contrast Ratio):

- Tło: **#D4AF37** (Luminancja względna: ~0.46)
- Tekst: **#004D40** (Luminancja względna: ~0.07)
- **Wynik:** Stosunek wynosi około **4.8:1**.
- **Wniosek:** Spełnia to wymóg **WCAG 2.1 Level AA** dla normalnego tekstu (wymagane 4.5:1) oraz zbliża się do wymogu AAA dla tekstu dużego. Biorąc pod uwagę, że używamy wagi SemiBold (która jest traktowana łagodniej w normach) oraz relatywnie dużych rozmiarów fontu, jest to zestawienie bezpieczne i zgodne z prawem (np. Section 508).

3.2. Kolory Secondary: Złoto i Fiolet

Wymaganie „obramowanie złote/fioletowe” sugeruje istnienie dwóch pod-wariantów przycisku drugorzędneho.

- **Secondary Gold (#D4AF37 Border):** Stosowany na ciemnych tłach (gdzie złoto świeci) lub jako uzupełnienie przycisku Primary na białym tle.
- **Secondary Purple (#6200EE Border):** Fiolet (Deep Purple) jest kolorem komplementarnym do złota w systemach triadycznych lub split-complementary.
 - Kolor tekstu w przyciskach Secondary: Zazwyczaj dziedziczy kolor obramowania, aby powiązać wizualnie granice z treścią. Zatem tekst będzie odpowiednio złoty (na ciemnym tle) lub fioletowy. Fioletowy tekst (#6200EE) na białym tle ma kontrast **10.8:1** (AAA), co jest doskonałym wynikiem.

3.3. Kolor Destructive (Błąd i Usuwanie)

Wymóg „odcień czerwony” musi zostać skonkretyzowany z uwzględnieniem psychologii błędu. Czysta czerwień (#FF0000) jest często zbyt agresywna i powoduje wibrację na ekranie. Badania i opinie ekspertów sugerują użycie odcieni z domieszką czerni lub błękitu, aby uczynić czerwień bardziej „profesjonalną” i mniej „alarmującą” w sposób techniczny. Rekomendacja: **#B00020** (Standard Error Red w Material Design) lub **#C62828**. Dla przycisku Destructive (zazwyczaj outline lub jasne tło):

- Tło: #FEECEB (Bardzo rozbielona czerwień).
- Tekst/Border: #B00020. Takie połączenie komunikuje destrukcyjność akcji, ale nie krzyczy na użytkownika, zachowując elegancję systemu.

3.4. Adaptacja do Trybu Ciemnego (Dark Mode)

System kolorystyczny musi być elastyczny. W trybie ciemnym (Dark Mode), duże powierzchnie nasyconego złota (#D4AF37) mogą powodować nadmierne zmęczenie wzroku (eye strain). Zalecana modyfikacja :

- Desaturacja kolorów wiodących. Złoto powinno stać się jaśniejsze i mniej nasycone (np. #FDD835), aby zmniejszyć kontrast wibracyjny z ciemnoszarym tłem (#121212).
- Kolor „Error” w trybie ciemnym powinien ewoluować w stronę pastelowego różu (#CF6679), ponieważ ciemna czerwień jest słabo widoczna na czarnym tle.

4. Geometria i Skalowanie: System Siatki 8-punktowej

Wymiary przycisków nie mogą być dziełem przypadku. Wymagane wysokości (56px, 48px, 40px) idealnie wpisują się w **8-point Grid System**, standard branżowy zapewniający rytm i harmonię wizualną.

4.1. Tabela Specyfikacji Wymiarowej

Poniższa tabela definiuje parametry fizyczne dla każdego rozmiaru, zapewniając spójność implementacji.

Rozmiar (Size)	Wysokość (Height)	Padding Poziomy (X)	Stopień Pisma (Font Size)	Ikona (Icon Size)	Zastosowanie (Use Case)
Large (L)	56px	32px	18px (1.125rem)	24px	Główne CTA, Landing Pages, Formularze Logowania. Idealny cel dotykowy.
Medium (M)	48px	24px	16px (1.0rem)	20px	Standardowy przycisk w aplikacjach, kartach produktów, oknach modalnych.
Small (S)	40px	16px	14px (0.875rem)	16px	Filtry, gęste tabele danych, akcje drugorzędne w nagłówkach.

4.2. Promień Zaokrąglenia (Border-Radius 8px)

Wymóg „umiarkowanego zaokrąglenia 8px” pozycjonuje styl jako „Friendly Modern”.

- **Analiza Psychologiczna:** Kąty proste (0px) są postrzegane jako techniczne, surowe i poważne. Pełne zaokrąglenia (Pill/Capsule, 50%) są postrzegane jako zabawowe, miękkie i mobilne. Wartość 8px to „złoty środek” – wystarczająco miękki, by zapraszać do interakcji (affordance), ale wystarczająco stabilny, by budować zaufanie w kontekście finansowym czy biznesowym.
- **Spójność Krzywizn:** Ważne jest, aby promień 8px był stosowany konsekwentnie również w elementach zagnieżdżonych. Np. focus ring (obrys fokusa) powinien mieć promień odpowiednio większy (8px + gap + border), aby zachować koncentryczność krzywych.

4.3. Obszar Dotykowy (Touch Target)

Należy zauważyć, że nawet najmniejszy przycisk (40px) jest poniżej zalecanego przez Apple (44pt) i WCAG 2.1 (44px dla AAA) minimalnego obszaru dotykowego.

- **Rozwiązanie:** W implementacji CSS należy zastosować pseudoelementy (::before lub ::after) zwiększające niewidzialny obszar klikalny do minimum 48x48px, nie zmieniając wizualnego rozmiaru przycisku. Jest to kluczowe dla użyteczności na urządzeniach mobilnych.

5. Architektura Stanów Interakcji: Choreografia UI

Projektowanie przycisku to projektowanie reakcji na działania użytkownika. Statyczny obraz przycisku to tylko wierzchołek góry lodowej. Prawdziwe UX dzieje się w czasie. Zestaw wymaga uwzględnienia wszystkich stanów: Default, Hover, Active, Focus, Disabled, Loading.

5.1. State: Hover (Najazd Kursorem)

Stan hover informuje użytkownika, że element jest interaktywny.

- **Mechanizm:** Zamiast prostej zmiany koloru hex na inny hex, co jest trudne w utrzymaniu, zaleca się manipulację jasnością (Luminance) lub kryciem (Opacity) warstwy wierzchniej.
- **Dla Złotego Primary:** Ponieważ złoto jest jasne, przyciemnienie go (Darken) o 10-15% daje efekt „gęstości” i solidności. Alternatywnie, rozjaśnienie (Lighten) sugeruje „podświetlenie”. Rekomenduję **przyciemnienie**, aby zwiększyć kontrast tekstu przy najechaniu.
- **Dla Secondary (Outline):** Wypełnienie przezroczystego tła kolorem bazowym (złotym lub fioletowym) o bardzo niskim kryciu (np. 10% opacity: rgba(212, 175, 55, 0.1)). Daje to subtelny, elegancki efekt.

5.2. State: Active (Pressed)

Moment fizycznego kliknięcia. Interfejs musi dać odczucie taktylne.

- **Skala:** Zastosowanie transformacji scale(0.98) symuluje fizyczne wciskanie przycisku w głąb ekranu.
- **Cień:** Jeśli przycisk posiada cień (elevation), w stanie Active powinien on zniknąć lub znacznie się zmniejszyć, co wzmacnia iluzję nacisku.

5.3. State: Focus (Dostępność Klawiaturowa)

Wymóg: „wyraźny outline”. Jest to krytyczne dla użytkowników nawigujących klawiaturą (Tab key).

- **Problem:** Standardowy niebieski obrys przeglądarki (User Agent Stylesheet) często jest niewidoczny na złotym lub turkusowym tle.
- **Rozwiązanie (Double Ring):** Zastosowanie podwójnego obrysu z odstępem (offset).
 1. Odstęp (Offset): 2px przezroczystości (ukazuje tło strony).
 2. Obrys (Outline): 2-3px solidnej linii w kolorze **Fioletowym Secondary** (#6200EE) lub **Ciemnoturkusowym** (#004D40). To rozwiązanie gwarantuje kontrast niezależnie od tego, czy przycisk znajduje się na białym, czy ciemnym tle.

5.4. State: Disabled (Wyłączony)

Stan ten komunikuje, że akcja jest niemożliwa.

- **Wizualizacja:** Tło zmienia się na neutralną szarość (np. #E0E0E0), a tekst na ciemniejszą szarość (#9E9E9E).
- **Uwaga A11y:** Elementy disabled są domyślnie ignorowane przez czytniki ekranowe i klawiaturę. Jednakże, dobra praktyka UX sugeruje czasem pozostawienie elementu fokusowalnego, aby czytnik mógł przeczytać tooltip wyjaśniający, *dlatego* przycisk jest nieaktywny.
- **Stylizacja Secondary Disabled:** Border staje się szary, tło pozostaje przezroczyste.

5.5. State: Loading (Ładowanie) i Problem CLS

To najbardziej złożony stan techniczny. Wymóg „spinner” musi być zrealizowany bez naruszania

struktury strony.

- **Zagrożenie CLS (Cumulative Layout Shift):** Jeśli podczas ładowania tekst „Zatwierdź” znika i jest zastępowany węższym spinnerem, szerokość przycisku maleje. Powoduje to przesunięcie sąsiednich elementów. Jest to błąd kardynalny karany przez Google w rankingu SEO i irytujący dla użytkownika.
- **Rozwiązanie Implementacyjne (Grid Stacking):** Najlepszą metodą jest wykorzystanie CSS Grid.
 - Przycisk jest kontenerem Grid z jedną komórką.
 - Tekst i Spinner znajdują się w tej samej komórce (nakładają się na siebie).
 - W stanie normalnym: Tekst opacity: 1, Spinner opacity: 0.
 - W stanie loading: Tekst opacity: 0 (ale wciąż zajmuje miejsce!), Spinner opacity: 1. Dzięki temu przycisk zachowuje swoje wymiary piksel w piksel, niezależnie od treści.
- **Spinner Style:** Spinner powinien być minimalistycznym okręgiem SVG z animacją stroke-dasharray, w kolorze tekstu przycisku (np. Ciemnoturkusowy na Żółtym), aby zachować spójność.

6. Warianty Funkcjonalne i Semantyka

System przycisków to nie tylko jeden „Master Button”, ale rodzina wariantów dostosowanych do kontekstu.

6.1. Warianty Secondary – Kontekst Użycia

Wymóg posiadania obramowania Żółtego lub Fioletowego wymaga jasnych reguł użycia (Governance), aby uniknąć chaosu.

- **Secondary Gold (Żółty Border):** Powinien być używany w kontekście „Premium” lub „Brand”. Np. akcja „Zobacz ofertę VIP”, „Dołącz do klubu”. Jest to most pomiędzy głównym CTA a tłem.
- **Secondary Purple (Fioletowy Border):** Powinien być używany dla akcji funkcjonalnych, technicznych lub związanych z procesami. Np. „Edytuj profil”, „Konfiguracja”, „Więcej opcji”. Fiolet w psychologii koloru kojarzony jest z kreatywnością i mądrością, co pasuje do akcji konfiguracyjnych.

6.2. Wariant: Icon Button (Z Ikoną)

Ikony przyspieszają rozpoznawanie funkcji (kognitywne przetwarzanie obrazu jest szybsze niż tekst).

- **Pozycjonowanie:** Ikona zazwyczaj znajduje się po lewej stronie tekstu (Leading). Dla akcji oznaczających ruch w przód (np. „Dalej”, „Przejdź do kasy”), ikona strzałki powinna być po prawej (Trailing).
- **Odstępy:** Zgodnie z siatką 8pt, odstęp między ikoną a tekstem powinien wynosić dokładnie **8px**.
- **Wyrównanie Optyczne:** Mukta Malar ma specyficzną wysokość x (x-height). Należy zadbać, aby środek geometryczny ikony pokrywał się ze środkiem optycznym wielkiej litery tekstu. Często wymaga to mikro-korekty CSS (np. `translateY(-1px)`).

6.3. Wariant: Full Width (Pełna Szerokość)

Krytyczny dla urządzeń mobilnych (Responsive Web Design).

- **Behavior:** Przycisk rozciąga się na 100% szerokości kontenera rodzica (width: 100%).
- **Zastosowanie:** W dolnych paskach nawigacyjnych (Sticky Bottom Bar) na smartfonach.
- **Stylizacja:** Często w tym wariantie rezygnuje się z zaokrąglenia (border-radius: 0), jeśli przycisk dotyka krawędzi ekranu, jednakże przy założeniu „border-radius 8px” i marginesach bocznych (np. 16px od krawędzi ekranu), przycisk zachowuje swoją „kapsułkową” formę, co jest bardziej nowoczesnym podejściem (tzw. Floating Action Button style).

7. Specyfikacja Techniczna i Implementacja (Kod)

Poniższa sekcja zawiera gotowe do wdrożenia wytyczne dla programistów CSS/SCSS, realizujące wszystkie powyższe założenia teoretyczne.

7.1. Architektura CSS (Design Tokens)

Zaleca się użycie zmiennych CSS (Custom Properties) dla łatwej modyfikacji i obsługi trybów kolorystycznych.

```
:root {  
  /* Paleta Kolorów */  
  --btn-color-primary-bg: #D4AF37;           /* Metallic Gold */  
  --btn-color-primary-fg: #004D40;           /* Dark Turquoise */  
  --btn-color-secondary-gold: #D4AF37;  
  --btn-color-secondary-purple: #6200EE;  
  --btn-color-destructive: #B00020;  
  
  /* Typografia */  
  --btn-font-family: 'Mukta Malar', sans-serif;  
  --btn-font-weight: 600;  
  
  /* Geometria */  
  --btn-radius: 8px;  
  
  /* Cienie */  
  --btn-shadow-rest: 0 2px 4px rgba(0,0,0,0.1);  
  --btn-shadow-hover: 0 4px 8px rgba(0,0,0,0.15);  
  
  /* Animacje */  
  --btn-transition: 200ms cubic-bezier(0.4, 0, 0.2, 1);  
}
```

7.2. Struktura Komponentu (Rozwiązanie Grid Stacking)

Rozwiązanie problemu Loading State w kodzie:

```
<button class="btn btn--primary btn--large" aria-busy="false">
  <span class="btn__content">
    <svg class="btn__icon">...</svg> Zapisz Zmiany
  </span>
  <span class="btn__spinner" aria-hidden="true">
    <svg class="spinner">...</svg>
  </span>
</button>

/* Struktura CSS */
.btn {
  /* Layout */
  display: inline-grid;
  grid-template-areas: "stack"; /* Klucz do sukcesu */
  place-items: center;

  /* Wymiary i Styl */
  height: var(--height); /* Zależne od wariantu S/M/L */
  padding: 0 var(--padding-x);
  border-radius: var(--btn-radius);
  border: 2px solid transparent; /* Rezerwacja miejsca na border */

  /* Typografia */
  font-family: var(--btn-font-family);
  font-weight: var(--btn-font-weight);

  /* Interakcja */
  cursor: pointer;
  transition: all var(--btn-transition);
  position: relative;
  overflow: hidden; /* Dla efektów ripple */
}

/* Stacking Context */
.btn__content,
.btn__spinner {
  grid-area: stack; /* Oba elementy w tym samym miejscu */
  display: flex;
  align-items: center;
  gap: 8px;
  transition: opacity 0.2s;
}

/* Domyślny stan spinnera */
.btn__spinner {
  opacity: 0;
  visibility: hidden;
}
```

```

}

/* Stan Loading */
.btn[aria-busy="true"] {
  cursor: wait;
}

.btn[aria-busy="true"].btn__content {
  opacity: 0; /* Ukryj tekst, ale zachowaj wymiary! */
  visibility: hidden;
}

.btn[aria-busy="true"].btn__spinner {
  opacity: 1;
  visibility: visible;
}

```

Taka konstrukcja kodu zapewnia absolutną stabilność układu. Przycisk nie drgnie nawet o piksel podczas przejścia w stan ładowania.

7.3. Optymalizacja Wydajności (Performance)

- **CSS Containment:** Użycie właściwości `contain: layout paint`; dla przycisku może pomóc przeglądarce w optymalizacji renderowania, informując, że zmiany wewnątrz przycisku nie wpływają na resztę strony.
- **Will-change:** Należy unikać nadużywania `will-change`. Dla prostych transformacji `scale` i opacyty przeglądarki radzą sobie doskonale bez tego.
- **Debounce Spinnera:** Z perspektywy JavaScript, należy opóźnić pojawienie się stanu `loading` o około 200-300ms. Jeśli akcja API wykona się szybciej (np. w 100ms), użytkownik nie zobaczy mignięcia spinnera, co jest postrzegane jako szybsze działanie systemu.

8. Podsumowanie i Wnioski

Przedstawiony raport definiuje kompletny ekosystem przycisku, który wykracza poza zwykłą definicję stylów. Poprzez połączenie **złotej estetyki**, **humanistycznej typografii Mukta Malar** i **inżynierskiego podejścia do dostępności i wydajności**, uzyskujemy komponent, który jest:

1. **Luksusowy i Spójny:** Dzięki precyzyjnie dobranej palecie #D4AF37 / #004D40 i typografii SemiBold.
2. **Dostępny:** Spełniający normy WCAG AA/AAA, z wyraźnym focussem i odpowiednim kontrastem.
3. **Stabilny:** Dzięki technice Grid Stacking eliminującej przesunięcia układu (CLS) podczas ładowania.
4. **Skalowalny:** Dzięki oparciu o siatkę 8-punktową i tokeny CSS.

Implementacja tego systemu zapewni aplikacji interfejs klasy premium, który nie tylko wygląda nowocześnie, ale działa w sposób przewidywalny i solidny, budując zaufanie użytkownika przy każdej interakcji. Rekomenduje się, aby niniejsza specyfikacja stanowiła podstawę (Foundation)

dla budowy Design Systemu w narzędziach takich jak Figma oraz w bibliotekach komponentów (React/Vue/Angular).

Cytowane prace

1. Mukta Malar - Google Fonts, <https://fonts.google.com/specimen/Mukta+Malar>
2. Preventing Layout Shifts: Mastering CSS Transitions and Animations - PixelFreeStudio Blog, <https://blog.pixelfreestudio.com/preventing-layout-shifts-mastering-css-transitions-and-animations/>
3. How To Avoid Large Layout Shifts: 3 Practical Examples - DebugBear, <https://www.debugbear.com/blog/avoid-large-layout-shifts>
4. Mukta is a Unicode compliant, contemporary, mono-linear font family available in seven weights, supporting Devanagari, Gujarati, Gurmukhi, Tamil and Latin scripts. - GitHub, <https://github.com/EkType/Mukta>
5. Modern Tamil Unicode Font - Google Groups, <https://groups.google.com/g/bvparishat/c/PKqEkjhSfME>
6. Mukta Malar | Adobe Fonts, <https://fonts.adobe.com/fonts/mukta-malar>
7. Mukta Malar by Google Fonts | Typographer, <https://typographer.com/fonts/gf-mukta-malar/>
8. @fontsource/mukta-malar - npm, <https://www.npmjs.com/package/@fontsource/mukta-malar>
9. Mukta Family - Ek Type, <https://ektype.in/font-mukta-family.html>
10. Making Color Usage Accessible | Section508.gov, <https://www.section508.gov/create/making-color-usage-accessible/>
11. Color | Digital Accessibility, <https://dap.berkeley.edu/web-a11y-basics/color>
12. Web Accessibility: Understanding Colors and Luminance - MDN Web Docs, https://developer.mozilla.org/en-US/docs/Web/Accessibility/Guides/Colors_and_Luminance
13. What is the most 'friendly' red hexadecimal color value for error messages? - Quora, <https://www.quora.com/What-is-the-most-friendly-red-hexadecimal-color-value-for-error-messages>
14. Dark theme - Material Design, <https://m2.material.io/design/color/dark-theme.html>
15. How do you make a button that does not resize on load? - AS Agency, <https://www.theasagency.com/en/blog/how-do-you-make-a-button-that-does-not-resize-on-load>
16. How to make page-load spinner spin smoothly? - Stack Overflow, <https://stackoverflow.com/questions/21183936/how-to-make-page-load-spinner-spin-smoothly>
17. Should a "loading" text or spinner stay a minimum time on screen? - UX Stack Exchange, <https://ux.stackexchange.com/questions/104606/should-a-loading-text-or-spinner-stay-a-minimum-time-on-screen>